

## Aula 2 — O relógio e o ciclo de busca-execução

*Onde a lógica booleana vira computação*

Microcontroladores — DENE/UFMT

### OBJETIVOS

- Justificar por que um circuito puramente combinacional só pode ser tratado como função booleana sob uma disciplina temporal, e distinguir essa disciplina do relógio que a implementa.
- Relacionar  $\tau_{cy}$ ,  $\tau_{osc}$  e a frequência do oscilador no PIC18F4550, e explicar de onde vem o fator quatro.
- Descrever o pipeline de dois estágios e derivar dele o custo em ciclos do desvio tomado, da instrução de duas palavras e do salto condicional sobre ela.
- Explicar por que o contador de programa tem o bit menos significativo fixo em zero, e por que o reset é em 0x0000 mesmo quando o arquivo HEX começa depois.
- Calcular o período do atraso escrito no encontro 0 e comparar com o palpite registrado então.

### 1 Duas dívidas

O encontro 0 deixou duas coisas por pagar. A primeira é um número: o período do pisca, que você ajustou por tentativa porque não tinha como calcular. A segunda é uma regra decorada: escrever LAT antes de TRIS.

As duas são a mesma dívida, e ela se chama tempo.

### 2 Por que existe um relógio

#### 2.1 A pedra e a idealização

Um somador de oito bits é um circuito puramente combinacional: dadas as entradas, a saída é uma função booleana delas. Essa frase é verdadeira e é uma idealização.

Troque as entradas de um somador ripple-carry e observe a saída com resolução suficiente. Antes de estabilizar no resultado correto, ela passa por uma sequência de valores espúrios, enquanto o vai-um se propaga de um estágio ao seguinte. Durante alguns nanossegundos o somador exhibe números que não são a soma de nada.

“Circuito de lógica booleana” descreve o regime permanente, não o transitório. O que torna a descrição booleana verdadeira **nos instantes que importam** é o ato de amostrar a saída depois que ela estabilizou, e de ignorá-la no resto do tempo.

CONCEITO

O relógio não organiza os passos apenas. Ele é o que dá licença para tratar o circuito como função.

#### 2.2 O que é realmente indispensável

Não é o relógio. São duas coisas: **estado**, para que o resultado de um passo sobreviva até o próximo, e **um critério de quando o estado é válido**.

O relógio é a forma mais comum de fornecer esse critério — uma referência global, distribuída por difusão, que declara periodicamente “agora pode amostrar”. Não é a única. Circuitos assíncronos resolvem o mesmo problema com protocolos de confirmação locais entre blocos vizinhos; o AMULET, um ARM sem relógio global, foi construído e funcionou.

Vale reter a distinção porque ela volta duas vezes. No encontro 11, o I<sup>2</sup>C confirma cada byte com ACK em vez de esperar um tempo fixo — é disciplina temporal por confirmação, dentro de um sistema síncrono. E nos seminários, o STM32 desliga o relógio de perifé-

NOTA

ricos ociosos e passa longos intervalos sem tique nenhum, o que seria impossível se o relógio fosse a essência da coisa.

### 2.3 O argumento que fecha

Um circuito combinacional soma dois números de oito bits. Para somar dois de dezesseis, é preciso **outro circuito**, maior. Uma família de circuitos, um por tamanho de problema.

Uma máquina não é assim. Ela é uma descrição finita — o mesmo somador de oito bits, os mesmos registradores — que serve para todos os tamanhos, porque reusa o mesmo bloco em instantes diferentes. A computação começa quando você para de fabricar pedra nova e passa a reusar a mesma pedra no tempo.

Daí a terminologia que este curso adota: chamamos um bloco combinacional de **circuito eletrônico de lógica booleana**, e reservamos **sistema digital computacional** para o que tem estado e disciplina temporal. As pedras são necessárias e não são suficientes. Tudo que vem a seguir nesta aula é o mecanismo que organiza as pedras no tempo neste chip específico.

CONCEITO

## 3 O relógio do PIC18F4550

### 3.1 O fator quatro

O oscilador entrega um sinal de frequência  $F_{osc}$ . O processador divide esse sinal em quatro fases não sobrepostas, Q1 a Q4, e o conjunto das quatro é um **ciclo de instrução**.

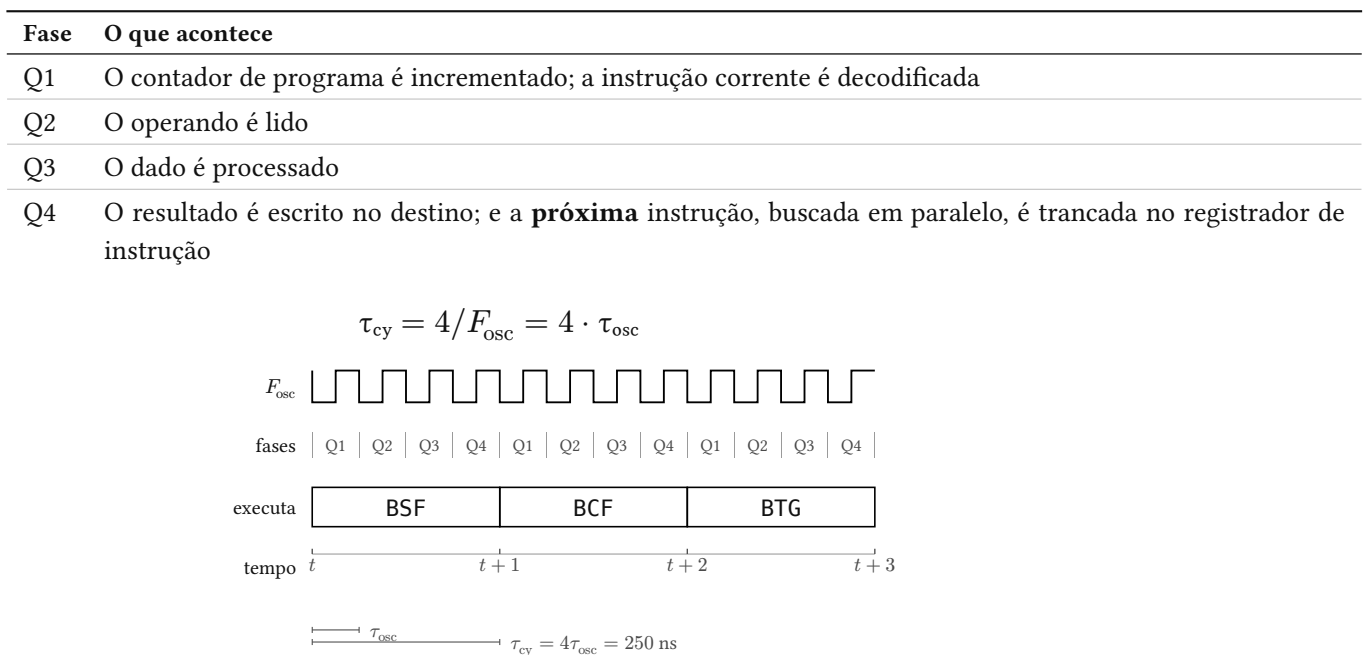


Figura 1: O oscilador não para; o que existe em número inteiro é o **ciclo de instrução**. Entre  $t$  e  $t+1$  o processador não está em lugar nenhum intermediário — é essa granularidade que torna possível contar tempo somando ciclos.

| $F_{osc}$ | $\tau_{osc}$ | $\tau_{cy}$   | Onde aparece                                       |
|-----------|--------------|---------------|----------------------------------------------------|
| 16 MHz    | 62,5 ns      | <b>250 ns</b> | O núcleo deste kit. Todo cálculo do semestre       |
| 48 MHz    | 20,8 ns      | 83,3 ns       | O que o módulo USB exige, e não o que o núcleo usa |
| 8 MHz     | 125 ns       | 500 ns        | Oscilador interno, sem cristal                     |

O 250 ns não é uma escolha sua. Quem gravou a configuração foi o bootloader, e ele fixou 48 MHz para a USB e 16 MHz para o núcleo. Seus `#pragma config` são ignorados — isso é do encontro 0, e é a razão de o número ser o mesmo em todas as bancadas.

ATENÇÃO

### 3.2 Por que quatro e não um

Cada fase existe para separar operações que não podem acontecer ao mesmo tempo sobre o mesmo barramento: ler um operando e escrever um resultado no mesmo instante exigiria dois barramentos de dados. Quatro fases é o preço de ter um só.

A consequência é que o processador executa a 4 MHz de instruções enquanto o cristal oscila a 16 MHz — e a razão pela qual esse número não é ainda pior é o assunto da seção seguinte.

## 4 O pipeline de dois estágios

A memória de programa e a memória de dados são separadas (encontro 1), com barramentos separados. Logo, buscar a próxima instrução não disputa nada com executar a atual, e as duas coisas podem se sobrepor:

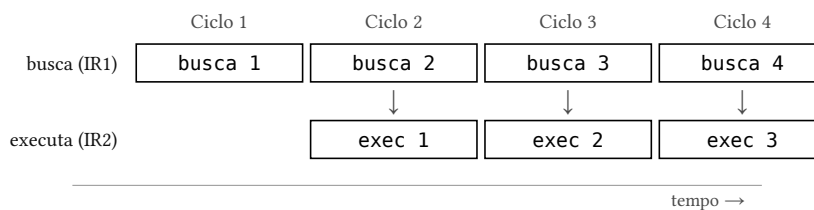


Figura 2: Em regime permanente. Só o primeiro ciclo tem o estágio de execução vazio.

Em regime, uma instrução termina por ciclo. Não porque ela leve um ciclo, mas porque as etapas de instruções vizinhas estão empilhadas.

Todo custo anômalo de ciclos neste processador tem a mesma causa: **a busca adiantada foi jogada fora**. Não são três regras diferentes a decorar; é uma regra com três formas.

CONCEITO

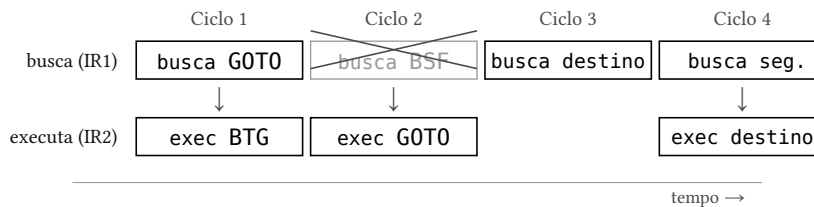


Figura 3: GOTO custa dois ciclos. O BSF seguinte já havia sido buscado e é descartado; o ciclo 3 busca o destino e não executa nada. O buraco no fluxo de execução é o ciclo extra.

| Situação                                                  | Ciclos | Por quê                                                                                              |
|-----------------------------------------------------------|--------|------------------------------------------------------------------------------------------------------|
| Instrução comum                                           | 1      | O pipeline está cheio e nada é descartado                                                            |
| Desvio tomado (GOTO, BRA, CALL, retorno)                  | 2      | A instrução já buscada não é a próxima. Ela é descartada e o ciclo seguinte é gasto buscando a certa |
| Instrução de duas palavras (GOTO, CALL, MOVFF, LFSR)      | 2      | A segunda palavra precisa ser buscada. Ela ocupa um ciclo de busca que não produz execução           |
| Salto condicional que pula uma instrução de duas palavras | 3      | Um ciclo pelo salto e mais um porque há duas palavras a atravessar                                   |

A segunda palavra de uma instrução de duas palavras tem os quatro bits mais significativos em 1111. Se o programa cair sobre ela por engano — por exemplo, um salto condicional que pula apenas uma palavra —, o processador a executa como uma instrução sem efeito, e não como o começo de outra coisa. É uma proteção de projeto, e é também a razão de o padrão 1111 estar reservado.

NOTA

## 5 O contador de programa

### 5.1 Vinte e um bits, e o último fixo

A Flash é endereçada por byte e a instrução tem 16 bits: toda instrução começa em endereço par. O contador de programa tem 21 bits com o menos significativo **fixo em zero** — ele não é um bit gravável, é um zero soldado.

Consequência: o PC anda de dois em dois, e não existe estado em que o processador esteja no meio de uma instrução.

21 bits alcançam 2 MB, e este chip tem 32 kB. A largura é da família, como os 12 bits do espaço de dados no encontro 1. Mesmo argumento, outro espaço.

NOTA

E é por isso que GOTO é uma instrução de duas palavras: 21 bits de destino não cabem numa palavra de 16 que ainda precisa carregar o código da operação.

### 5.2 O reset é 0x0000

O endereço de reset é fixo em silício. Não é uma convenção do compilador, não é um campo do arquivo HEX, e não é o menor endereço presente nele.

No kit, o bootloader ocupa o começo da Flash e o seu programa começa depois. Abra o HEX da sua aplicação e o menor endereço não será 0x0000.

DIVERGÊNCIA

Um simulador que comece a executar no menor endereço do HEX funciona perfeitamente com um programa autônomo e falha com um programa de bootloader — e falha **silenciosamente**, executando algo que não é o começo — e é um erro fácil de cometer, porque a versão errada funciona em todos os testes que não usam bootloader. O simulador deve começar em 0x0000 e encontrar lá o que estiver; se não houver nada, o certo é reclamar, não adivinhar.

## 6 O ciclo, escrito

### 6.1 De onde vem self.programa

Antes de executar é preciso carregar. O montador não entrega bytes soltos: ele entrega um arquivo de texto em que cada linha é um registro com contagem, endereço de carga, tipo, dados e soma de verificação.

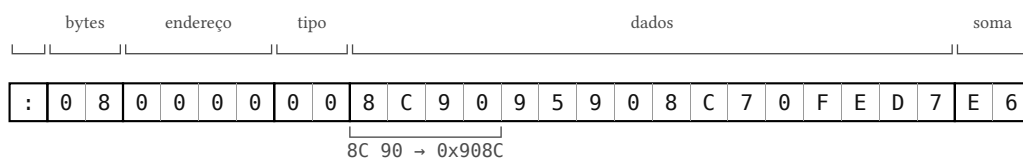


Figura 4: Um registro do arquivo HEX: o pisca inteiro em oito bytes. Contagem de bytes, endereço de carga, tipo de registro, dados e soma de verificação. O endereço é do **registro**, não do programa — é dele que sai a divergência da seção anterior. Aqui vale 0x0000; gravado pelo bootloader, o primeiro registro da aplicação começa em 0x0800.

Carregar é copiar os bytes de cada registro para o endereço que o próprio registro declara. Nada mais. O simulador que ignora esse campo e empilha os bytes na ordem em que os lê funciona por acidente enquanto houver um registro só.

### 6.2 O laço

Com o espaço de dados do R2 pronto e o HEX carregado, o núcleo do simulador cabe em cinco linhas:

```
while True:
    palavra = self.programa[self.pc] | (self.programa[self.pc + 1] << 8)
    self.pc += 2
    self.executa(palavra)
```

Onde está o relógio nesse código? Em lugar nenhum. O `while` do Python é o relógio: o simulador está tomando emprestada a ordem temporal do interpretador, e por isso não há nada a escrever para “ligar o oscilador”.

DIVERGÊNCIA

A consequência é que o simulador executa depressa e **não sabe que horas são** — ele reproduz a ordem dos passos e perde a duração deles. Como toda esta aula é sobre duração, a correção é obrigatória e é barata:

```
self.ciclos += n          # n vem da tabela da §4
tempo = self.ciclos * 250e-9
```

No simulador, o relógio é um acumulador. Na bancada, é um sinal físico que você não consegue enxergar diretamente, mas consegue **pesar**: mede-se um laço de ciclos contados e deduz-se a frequência do oscilador. É o R3.

## 7 O pino é só um endereço

Com seis instruções — `BSF`, `BCF`, `MOVLW`, `MOVWF`, `GOTO`, `NOP` — já se pisca um LED. `LATD` está em `0xF8C` (encontro 1), e escrever ali move elétrons num fio que sai do encapsulamento.

```

ORG      0x0000
BCF      0x8C, 0, 0      ; LATD0 = 0    -- 1 ciclo, access bank
BCF      0x95, 0, 0      ; TRISD0 = 0   -- 1 ciclo, o pino vira saída

laco:
BTG      0x8C, 0, 0      ; inverte o LED    -- 1
CALL     atraso          ;                  -- 2
BRA      laco            ; desvio tomado    -- 2
```

### 7.1 Uma palavra, decodificada

A primeira instrução da listagem é a palavra `0x908C`, que em binário é `1001 0000 1000 1100`. O conjunto de instruções divide esses dezesseis bits em quatro campos:

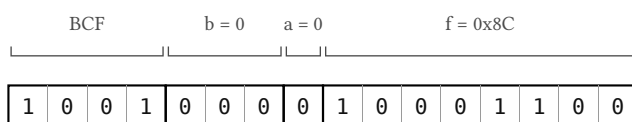


Figura 5: `BCF 0x8C, 0, 0` — zere o bit 0 do arquivo `0x8C`, usando o access bank. O campo `f` tem oito bits e o access bank o resolve sem `BSR` (encontro 1); o campo `b` tem três, que é exatamente o que se precisa para escolher um bit entre oito.

Nada nessa palavra menciona um LED, um pino ou uma porta. O que existe é um endereço, um bit dentro dele e uma operação. O LED é consequência de `0x8C` ser `LATD` e de `LATD0` sair no pino 19.

### 7.2 LAT antes de TRIS, agora como consequência

Inverta as duas primeiras linhas e pergunte o que o pino faz entre elas.

Com `TRIS` zerado primeiro, o pino passa a **dirigir** o barramento no instante seguinte, e o valor que ele dirige é o que estiver em `LATD` — que, depois do reset, a folha de dados declara indefinido. O pino sai com um valor que ninguém escolheu, e continua assim até a escrita em `LAT`.

| Caso           | Duração do intervalo em que o pino dirige valor indefinido                                                                                 |
|----------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| Neste exemplo  | Um ciclo. 250 ns de valor indefinido no pino                                                                                               |
| Em código real | O intervalo é o que houver entre as duas escritas: uma chamada de função, uma inicialização de periférico, um laço. Pode ser milissegundos |

250 ns não movem um relé nem acendem um LED de forma perceptível. Mas o raciocínio não é sobre 250 ns: é sobre o intervalo **que você não controla** quando

CONCEITO

as duas escritas ficam longe uma da outra. Um estágio de potência com entrada rápida obedece a qualquer coisa que apareça no pino, e o aquecedor do encontro 8 não pergunta se o valor era intencional.

A regra do encontro 0 era “escreva LAT antes de TRIS”. A regra agora é “nenhum pino de saída deve ser habilitado antes que o seu valor esteja decidido”, e a diferença entre as duas é que a segunda você consegue aplicar a um caso que não estava no exemplo.

## 8 Contagem de ciclos: pagando a primeira dívida

### 8.1 A tabela

| Instrução                  | Ciclos | Instrução                           | Ciclos |
|----------------------------|--------|-------------------------------------|--------|
| MOVLW, MOVWF, MOVF         | 1      | GOTO, CALL                          | 2      |
| BSF, BCF, BTG              | 1      | BRA, RCALL                          | 2      |
| INCF, DECF, CLRF           | 1      | RETURN, RETLW                       | 2      |
| NOP                        | 1      | BZ, BNZ, BC (tomados)               | 2      |
| DECFSZ, INCFSZ (sem pular) | 1      | BZ, BNZ, BC (não tomados)           | 1      |
| DECFSZ, INCFSZ (pulando)   | 2      | Salto sobre instrução de 2 palavras | 3      |

### 8.2 Um atraso contado

```

atraso:
    MOVLW    0x75
    MOVWF    cH, 0           ; 1
h:      CLRF    cL, 0         ; 1
l:      DECFSZ  cL, 1, 0      ; 1, ou 2 na última
        BRA     l             ; 2
        DECFSZ  cH, 1, 0      ; 1, ou 2 na última
        BRA     h             ; 2
        RETURN                ; 2

```

O laço interno custa DECFSZ + BRA = 3 ciclos por volta, 256 voltas, mais 1 ciclo extra na última passagem: 769 ciclos. O laço externo acrescenta CLRF mais DECFSZ mais BRA = 4 ciclos por volta, e roda 0x75 = 117 vezes.

$$117 \cdot (769 + 4) = 90441 \text{ ciclos} = 90441 \cdot 250 \text{ ns} \approx 22,6 \text{ ms}$$

(a) O maior atraso possível com **dois** laços de oito bits encaixados é de quanto, a 16 MHz?

TAREFA

(b) O pisca do encontro 0 precisava de meio segundo por semiperíodo. Quantos níveis de encaixe são necessários?

### 8.3 O modelo linear

Nenhum atraso é só o laço. Sempre há um custo fixo — a preparação dos contadores, a chamada, o retorno — que não escala com a contagem:

$$H = a + bN$$

O termo  $bN$  é o laço;  $a$  é tudo o mais. Dobrar  $N$  só dobra  $H$  quando  $a \ll bN$ , que é a suposição que você fez no encontro 0 sem saber que estava fazendo. Ela é boa para  $N$  grande e ruim para  $N$  pequeno — e a separação entre os dois casos vira diagnóstico no R3.

Duas formas de erro de tempo, e cada uma tem uma assinatura diferente:

CONCEITO

**Erro proporcional**, igual em todas as escalas de tempo  $\rightarrow$  o valor de  $b$  está errado. Constante de conversão, frequência de oscilador, número de ciclos por volta.

**Erro que cresce quando o intervalo diminui**  $\rightarrow a$  não é desprezível. Uma sobrecarga fixa vale proporcionalmente mais quando o intervalo é curto.

Diagnostique pela **forma** dos números, nunca pelo sinal ou pela magnitude de um deles isolado.

## 9 Previsão para o R3

### PREVISÃO

**P1.** Com a listagem do seu pisca em mãos, conte os ciclos de um semiperíodo e converta para milissegundos. Escreva o número antes de ligar o osciloscópio.

**P2.** Compare com o palpite que você registrou no encontro 0. De quanto errou, e para que lado?

**P3.** Se o valor medido vier **maior** que o contado, onde estão os ciclos que faltam na sua conta? Liste dois candidatos.

**P4.** Se o valor medido vier menor, o que isso diria sobre a hipótese de que o laço sobreviveu ao compilador?

**P5.** Você vai medir o mesmo sinal duas vezes, com a mesma base de tempo. Que diferença entre as duas medidas você consideraria ruído, e a partir de qual diferença você suspeitaria de um erro de método?

Leve esta folha preenchida, a listagem impressa ou aberta, e a contagem de ciclos já feita. Medir sem previsão transforma o roteiro em digitação.

BANCADA

## 10 Exercícios

**Exercício 2.1.** Um colega afirma: “meu somador é combinacional, então ele é um sistema digital; o relógio só serve para deixá-lo mais organizado.”

TAREFA

(a) Dê um instante específico em que a saída do somador não é a soma das entradas.

(b) Qual é a diferença entre uma família de circuitos e uma máquina?

(c) O relógio é indispensável para a computação digital? Justifique com um contraexemplo.

**Exercício 2.2.** Considere:

TAREFA

```
BTFSCL    0x80, 0, 0
GOTO      outro_lugar
BSF        0x8C, 1, 0
```

(a) Quantos ciclos o trecho consome quando o bit testado vale 1?

(b) E quando vale 0?

(c) O tempo de execução deste trecho depende de um dado externo. Que problema isso cria para um atraso escrito com laços contados que contenha um teste parecido dentro?

**Exercício 2.3.** Você grava uma aplicação pelo bootloader e abre o HEX: o menor endereço é 0x0800.

TAREFA

- (a) O reset ainda é em 0x0000?
- (b) O que existe em 0x0000?
- (c) Seu simulador começa a executar no menor endereço do HEX. Descreva o sintoma que isso produz, e por que ele é pior do que um erro que trava.

**Exercício 2.4.** Um atraso calibrado apresenta erro de  $-3\%$  em todos os intervalos que você mede, de 1 ms a 500 ms.

TAREFA

- (a) O defeito está em  $a$  ou em  $b$ ?
- (b) Que grandeza física poderia produzir exatamente esse padrão?
- (c) Como a assinatura mudaria se o defeito fosse uma sobrecarga fixa de 40 ciclos por volta do laço externo?

**No encontro 3:** barramento paralelo e o controlador HD44780. O display é o periférico mais lento do sistema, e a sua sequência de inicialização é uma sucessão de esperas cegas — o processador aguarda um tempo fixo porque não tem como perguntar se já acabou. A contagem de ciclos de hoje passa a ser a unidade em que se mede o custo de uma atualização de tela.

NOTA